
HEEC

École des Hautes Études Économiques Commerciales
et d'Ingénierie de Marrakech

**RAPPORT DE GÉNIE LOGICIEL
ET CONDUITE DE PROJET**

Application des concepts de Génie Logiciel
et de Conduite de Projet au développement de
SaaS Directory

Présenté par :

**Omar SARSAR
Iliass HARIZ**

2026



*Nous dédions ce travail à nos familles pour leur soutien indéfectible,
à nos enseignants pour leur guidance et leurs enseignements,
et à tous ceux qui croient en l'innovation technologique au service de la
communauté.*

Nous tenons à exprimer notre profonde gratitude envers l'ensemble de nos enseignants qui nous ont accompagnés tout au long de notre parcours académique. Leurs conseils avisés, leur patience et leur exigence ont été des facteurs déterminants dans la réussite de ce projet.

Nous remercions également nos camarades de promotion pour les échanges enrichissants et l'entraide collective qui ont contribué à notre progression.

Ce rapport présente l'application systématique des concepts de Génie Logiciel et de Conduite de Projet au développement de **SaaS Directory**, une plateforme de marketplace dédiée aux produits logiciels en mode SaaS. Le projet a été réalisé avec une stack technologique moderne comprenant Next.js 16.2.4, React 19.2.4, TypeScript, Tailwind CSS v4, Drizzle ORM 0.45.2, PostgreSQL et Better Auth 1.6.9.

L'étude couvre l'ensemble des neuf chapitres du cours de Génie Logiciel : rappels fondamentaux, généralités, cycle de vie du logiciel, démarche de conception, normalisation, tests, gestion de projet, analyse des risques et conduite du changement. Chaque concept théorique est illustré par sa mise en œuvre concrète dans le projet.

Le rapport inclut une estimation du coût par la méthode COCOMO appliquée à la base de code réelle de 12 166 lignes, une analyse des risques liés aux breaking changes de Next.js 16.2.4, ainsi qu'une présentation détaillée de la gestion des versions via Git et des migrations Drizzle ORM.

Mots-clés : Génie Logiciel, Conduite de Projet, Next.js, React, Drizzle ORM, PostgreSQL, CO-COMO, Cycle de Vie, UML, Normalisation, Tests Logiciels, Gestion des Risques.

This report presents the systematic application of Software Engineering and Project Management concepts to the development of **SaaS Directory**, a marketplace platform dedicated to SaaS software products. The project was built using a modern technology stack including Next.js 16.2.4, React 19.2.4, TypeScript, Tailwind CSS v4, Drizzle ORM 0.45.2, PostgreSQL, and Better Auth 1.6.9.

The study covers all nine chapters of the Software Engineering course : fundamental recalls, general concepts, software life cycle, design methodology, standardization, testing, project management, risk analysis, and change management. Each theoretical concept is illustrated by its concrete implementation in the project.

The report includes a COCOMO cost estimation applied to the actual codebase of 12,166 lines, a risk analysis related to Next.js 16.2.4 breaking changes, and a detailed presentation of version control via Git and Drizzle ORM migrations.

Keywords : Software Engineering, Project Management, Next.js, React, Drizzle ORM, PostgreSQL, COCOMO, Life Cycle, UML, Standardization, Software Testing, Risk Management.

Remerciements	4
Résumé	5
Abstract	6
Introduction Générale	13
1 Chapitre 1 : Rappels	14
1.1 Définitions Fondamentales	14
1.1.1 Logiciel et Application	14
1.1.2 Système d'Information (SI)	14
1.1.3 Méthode, Outil et Méthodologie	15
1.2 Application à SaaS Directory	15
2 Chapitre 2 : Génie Logiciel à€” Généralités	15
2.1 Objectifs du Génie Logiciel	15
2.2 Les Principes du GL Appliqués	16
2.2.1 Rigueur	16
2.2.2 Séparation des Problèmes	16
2.2.3 Modularité	16
2.2.4 Abstraction	16
2.2.5 Généricité	16
2.2.6 Construction Incrémentale	17
2.2.7 Anticipation du Changement	17
2.3 Les Attributs du Logiciel	17
2.3.1 Couplage	17
2.3.2 Complexité	17
2.3.3 Extensibilité	18
2.3.4 Lisibilité et Visibilité du Comportement	18

3	Chapitre 3 : Représentations du Cycle de Vie du Logiciel	18
3.1	Les Modèles de Cycle de Vie	18
3.2	Choix et Justification du Modèle	19
3.3	Application au Projet SaaS Directory	19
3.3.1	Itération 1 : Fondations et Authentification	19
3.3.2	Itération 2 : Profil Utilisateur et Lancement de Produits	19
3.3.3	Itération 3 : Marketplace	19
3.3.4	Itération 4 : Messagerie et Notifications	20
3.3.5	Itération 5 : Monétisation	20
4	Chapitre 4 : La Démarche	20
4.1	Méthodes de Développement Semi-Formelles	20
4.2	Diagrammes UML Appliqués au Projet	21
4.2.1	Diagramme de Cas d’Utilisation	21
4.2.2	Diagramme de Classes	23
4.2.3	Diagramme de Séquence	23
4.3	Méthodes Formelles et Langage Z	24
5	Chapitre 5 : La Normalisation	25
5.1	La Norme ISO et la Certification ISO 9000	25
5.2	Les Normes IEEE	25
5.3	Normalisation Appliquée au Projet	26
5.3.1	Normes de Codage	26
5.3.2	Architecture Normalisée	26
5.3.3	Gestion Documentaire et Traçabilité	26
6	Chapitre 6 : Les Tests	26
6.1	Types de Tests et Stratégie	27
6.1.1	Tests Unitaires	27
6.1.2	Tests d’Intégration et End-to-End	27
6.1.3	Tests de Performance	27

6.2	Outils et Environnement de Test	27
6.3	Couverture et Résultats	28
7	Chapitre 7 : La Gestion des Projets Logiciels	28
7.1	Organisation du Projet (WBS, PBS, OBS)	28
7.2	Planification (GANTT, PERT)	28
7.3	Estimation des Coûts par COCOMO	29
7.3.1	Taille du Projet	29
7.3.2	Application du COCOMO de Base	30
8	Chapitre 8 : L'Analyse des Risques	30
8.1	Identification des Risques	30
8.1.1	Risques Techniques	30
8.1.2	Risques de Sécurité	31
8.1.3	Risques de Performance	31
8.2	Analyse et Stratégies de Mitigation	31
8.2.1	Mitigation des Risques Techniques	31
8.2.2	Mitigation des Risques de Sécurité	31
8.2.3	Mitigation des Risques de Performance	32
8.3	Plan de Continuité	32
9	Chapitre 9 : Conduite du Changement	32
9.1	Nature des Changements	32
9.2	Gestion des Versions	33
9.3	Adaptation et Communication	33
	Conclusion Générale et Perspectives	33
	Bibliographie et Webographie	35
	Annexes	36
	Annexe A : Schéma de la Base de Données	36

Annexe B : Structure du Projet	41
Annexe C : Dépendances Principales	44
Annexe D : Diagramme de Déploiement	45

1	Diagramme de Cas d'Utilisation â€” SaaS Directory	22
2	Diagramme de Classes â€” SaaS Directory (20 tables)	23
3	Diagramme de Séquence â€” Envoi d'un Message en Temps Réel	24
4	Diagramme de GANTT â€” Planification du Projet SaaS Directory	29
5	Diagramme de Déploiement â€” Architecture Physique de SaaS Directory	45

1	Estimation COCOMO de Base – Mode Semi-Détaché (14 KLOC)	30
2	Dépendances principales du projet SaaS Directory	44

Dans un monde de plus en plus numérisé, le développement de logiciels est devenu une discipline complexe qui exige rigueur, méthodologie et gestion efficace des ressources. Le Génie Logiciel (GL) et la Conduite de Projet fournissent le cadre théorique et pratique nécessaire pour mener à bien des projets logiciels de qualité, dans les délais et budgets impartis.

Ce rapport a pour objectif de démontrer l'application concrète des concepts théoriques du Génie Logiciel et de la Conduite de Projet au développement de **SaaS Directory**, une application web moderne permettant aux créateurs de logiciels de lancer, promouvoir et vendre leurs produits SaaS.

SaaS Directory est une application web moderne permettant aux créateurs de logiciels de lancer, promouvoir et vendre leurs produits SaaS (Software as a Service). L'application offre un marketplace où les utilisateurs peuvent découvrir des outils logiciels, voter pour leurs produits préférés, interagir via des commentaires et une messagerie temps réel, et souscrire à des plans d'abonnement Free ou Pro.

Le projet a été réalisé avec une stack technologique contemporaine et cohérente : Next.js 16.2.4 (framework React avec App Router), React 19.2.4, TypeScript 5, Tailwind CSS v4, Drizzle ORM 0.45.2 (ORM type-safe pour PostgreSQL), Better Auth 1.6.9 (authentification complète avec OAuth et 2FA), Redis (cache et Pub/Sub pour la messagerie temps réel), et Playwright 1.59.1 pour les tests end-to-end.

Le codebase compte exactement **20 tables** dans le schéma Drizzle ORM, **12 166 lignes de code** réparties sur **108 fichiers** TypeScript/TSX, avec **4 migrations** versionnées (0000 à 0003). Le projet est versionné sur Git avec une branche master principale et des branches de fonctionnalités.

Ce rapport s'articule autour de neuf chapitres principaux. Le premier chapitre présente les rappels fondamentaux sur les concepts de base. Le deuxième traite des généralités du Génie Logiciel et de ses principes. Le troisième décrit les modèles de cycle de vie. Le quatrième expose la démarche de conception avec UML. Le cinquième aborde la normalisation. Le sixième détaille les stratégies de tests. Le septième présente la gestion de projet et l'estimation COCOMO. Le huitième analyse les risques. Le neuvième et dernier chapitre traite de la conduite du changement.

1.1 Définitions Fondamentales

Avant d’aborder les concepts avancés du Génie Logiciel, il est essentiel de poser les bases terminologiques et conceptuelles sur lesquelles repose l’ensemble de la discipline.

1.1.1 Logiciel et Application

Un **logiciel** est défini comme un ensemble de programmes, procédures, règles et documentations relatifs au fonctionnement d’un ensemble de traitements d’informations. Cette définition englobe non seulement le code source, mais aussi la documentation technique, les manuels utilisateurs, et les spécifications fonctionnelles.

Une **application** est un logiciel qui sert à implanter un système d’informations. On distingue traditionnellement trois grandes catégories d’applications :

- **Applications de gestion** : caractérisées par peu de traitement et beaucoup de données (ex. CRM, ERP).
- **Applications scientifiques** : nécessitant beaucoup de traitement et peu de données (ex. simulations physiques).
- **Applications industrielles** : avec peu de traitement et peu de données (ex. systèmes embarqués).

SaaS Directory s’inscrit dans la catégorie des applications de gestion/web, avec une forte composante de traitement (messagerie temps réel, recherche full-text) et une volumétrie de données significative (20 tables relationnelles, millions de potentielles lignes).

1.1.2 Système d’Information (SI)

Le **Système d’Information (SI)** constitue l’ensemble des flux d’opérations, d’informations et de moyens mis en œuvre par une organisation pour traiter ses données et produire de l’information. L’entreprise est un système ouvert sur son environnement, échangeant des flux physiques (produits, personnel, argent) et des flux d’information.

Le **Système d’Information Automatisé (SIA)** est un système physique reposant sur la technologie informatique. C’est un ensemble de ressources matérielles et logicielles permettant de traiter automatiquement des informations. SaaS Directory est un SIA complet : il traite des données utilisateurs, des transactions de vote, des messages temps réel, et des abonnements, le tout via une infrastructure web moderne.

1.1.3 Méthode, Outil et Méthodologie

Une méthode est un processus discipliné qui génère un ensemble de modèles décrivant les différents aspects d'un système logiciel, en utilisant une notation bien définie.

Un **outil** désigne un programme ou un ensemble de programmes aidant à la mise en œuvre des techniques. L'**Atelier de Génie Logiciel (AGL)** regroupe l'ensemble des outils utilisés pour le développement, la maintenance et la documentation des logiciels.

La **méthodologie** est la démarche générale qui fait appel à des méthodes et des outils pour aboutir au logiciel final. Pour SaaS Directory, la méthodologie adoptée combine l'approche incrémentale itérative (développement par sprints fonctionnels de 3-4 semaines) avec les principes du cycle de vie en V (validation systématique à chaque phase).

1.2 Application à SaaS Directory

SaaS Directory s'inscrit parfaitement dans le cadre théorique des Systèmes d'Information Automatisés. En tant que marketplace web, il traite de grands volumes de données structurées (profils utilisateurs, produits, votes, messages, abonnements) et exige une architecture logicielle rigoureuse pour garantir performance, sécurité et scalabilité.

La méthodologie adoptée pour ce projet combine l'approche incrémentale itérative (développement par sprints fonctionnels) avec les principes du cycle de vie en V (validation systématique à chaque phase). Cette hybridation se justifie par la nature du projet : une application web critique nécessitant à la fois flexibilité (itérations rapides) et fiabilité (tests exhaustifs à chaque niveau).

2.1 Objectifs du Génie Logiciel

Le **Génie Logiciel (GL)** vise à utiliser l'informatique dans les entreprises comme outil quotidien, à réduire les coûts de développement et de maintenance, à améliorer la qualité des logiciels, et à augmenter la productivité des équipes. Ces objectifs s'inscrivent dans une démarche globale de maîtrise du cycle de vie du logiciel.

Le GL touche l'ensemble du cycle de vie du logiciel, depuis l'étude et l'analyse du besoin jusqu'à la maintenance en passant par la conception, le codage, les tests et la documentation. Cette vision holistique garantit que chaque phase est cohérente avec les autres et contribue à l'objectif final : un logiciel de qualité, livré dans les délais et respectant le budget.

2.2 Les Principes du GL Appliqués

Le cours de Génie Logiciel énonce sept principes fondamentaux. Leur application au projet SaaS Directory se décline comme suit.

2.2.1 Rigueur

La rigueur se traduit par une grande exactitude dans l'application des règles. Dans le projet, cette exigence est matérialisée par l'utilisation de TypeScript (typage statique strict), les conventions de nommage strictes (PascalCase pour les composants, camelCase pour les fonctions), et la revue systématique du code via les pull requests Git. Le fichier `AGENTS.md` documente les conventions spécifiques du projet et sert de référentiel pour maintenir la cohérence.

2.2.2 Séparation des Problèmes

Ce principe s'applique à trois niveaux dans SaaS Directory. La **séparation dans le temps** est illustrée par le cycle de développement en cinq itérations (fondations, profil, marketplace, messagerie, monétisation). La **séparation dans l'espace** est assurée par l'architecture en couches (Client Components dans Next.js 16.2.4, Server Actions, Drizzle ORM, PostgreSQL). La **séparation en sous-systèmes** se matérialise par la structuration du code en modules cohérents (`src/app/`, `src/components/`, `src/lib/`).

2.2.3 Modularité

La modularité est sans doute l'un des principes les plus visiblement appliqués dans le projet. L'architecture est fondée sur des composants React indépendants, chacun responsable d'une fonctionnalité spécifique. Le schéma Drizzle ORM décompose le système en 20 tables distinctes, chacune encapsulant une entité métier. Cette modularité facilite la maintenance, les tests unitaires, et l'évolution du système.

2.2.4 Abstraction

L'abstraction consiste à ne considérer à un moment donné que les objets jugés importants du système. SaaS Directory utilise plusieurs niveaux d'abstraction : les Server Actions abstraient la logique métier du rendu UI ; Drizzle ORM abstrait les requêtes SQL complexes ; le système de cache Redis abstrait la gestion des sessions et des données temporaires.

2.2.5 Généricité

La généricité est le fait pour un objet de pouvoir être utilisé tel quel dans différents contextes. Les composants UI du projet (boutons, cartes, formulaires) sont conçus comme des composants réutili-

sables paramétrables via les props React. Le système d’authentification Better Auth est générique et supporte OAuth (Google, GitHub) et l’authentification par email/mot de passe.

2.2.6 Construction Incrémentale

Le développement a strictement suivi une approche incrémentale :

1. **Itération 1** : mise en place Next.js 16.2.4 + authentification Better Auth + schéma Drizzle ORM
2. **Itération 2** : profil utilisateur + lancement de produits
3. **Itération 3** : marketplace (votes, commentaires, catégories)
4. **Itération 4** : messagerie temps réel + notifications
5. **Itération 5** : abonnements Free/Pro + monétisation

2.2.7 Anticipation du Changement

Ce principe préconise de prévoir et faciliter les changements en favorisant la construction modulaire. L’architecture de SaaS Directory anticipe plusieurs évolutions : le schéma Drizzle inclut déjà les champs `stripeCustomerId` et `stripeSubscriptionId` pour une future intégration Stripe ; la messagerie est conçue pour supporter facilement l’ajout de groupes ; le système de plans (Free/Pro) permet d’ajouter de nouveaux niveaux d’abonnement sans modifier le code existant.

2.3 Les Attributs du Logiciel

Les attributs du logiciel mesurent sa qualité intrinsèque. L’analyse du codebase de 12 166 lignes réparties sur 108 fichiers permet d’évaluer ces attributs objectivement.

2.3.1 Couplage

Le couplage mesure l’indépendance entre modules. Plus le couplage est faible, plus les modules sont indépendants. Dans SaaS Directory, le couplage est faible grâce à l’architecture par composants React et l’encapsulation des Server Actions. Chaque table Drizzle ORM est indépendante, avec des relations explicites via les clés étrangères. Le couplage entre le frontend (React) et le backend (Server Actions) est réduit au minimum par l’API interne de Next.js 16.2.4.

2.3.2 Complexité

Avec 20 tables relationnelles et 12 166 lignes de code, le système présente une complexité modérée. Cette complexité est maîtrisée par la modularité, le typage statique TypeScript, et la documentation

inline (JSDoc). Le ratio moyen de 112 lignes par fichier indique une bonne granularité des modules.

2.3.3 Extensibilité

L’extensibilité est le degré d’accommodation aux changements produits par une nouvelle exigence. Elle est démontrée par la facilité d’ajouter de nouvelles tables au schéma Drizzle (4 migrations versionnées), l’ajout de nouveaux composants UI sans impacter les existants, et la configuration modulaire de Next.js (`next.config.ts`).

2.3.4 Lisibilité et Visibilité du Comportement

La lisibilité est assurée par TypeScript (typage explicite), les conventions de nommage strictes, et la documentation JSDoc. La visibilité du comportement est garantie par les tests unitaires (11 fichiers de test couvrant l’authentification, la souscription, le schéma, les actions) et les tests end-to-end (Playwright 1.59.1).

Le développement d’un logiciel n’est pas simplement l’écriture d’un programme sur une durée variant de quelques heures à quelques années. La taille et la complexité des logiciels actuels exigent l’utilisation de processus de développement structurés, appelés **cycles de vie du logiciel**.

3.1 Les Modèles de Cycle de Vie

Plusieurs modèles de cycle de vie ont été définis dans la littérature du génie logiciel. Le modèle en cascade (waterfall) suit une séquence linéaire de phases : spécification, conception, codage, tests, et maintenance. Chaque phase doit être terminée avant de passer à la suivante. Ce modèle est rigide et peu adapté aux projets où les besoins évoluent.

Le modèle en V associe à chaque phase de développement une phase de test correspondante : spécification → tests d’acceptation, conception architecturale → tests système, conception détaillée → tests d’intégration, codage → tests unitaires. Ce modèle met l’accent sur la validation précoce.

Le modèle itératif et incrémental divise le projet en itérations courtes (sprints), chacune livrant une version fonctionnelle du logiciel. Chaque itération inclut analyse, conception, codage et tests. Ce modèle est particulièrement adapté aux projets web modernes où les besoins évoluent rapidement.

3.2 Choix et Justification du Modèle

Pour le projet SaaS Directory, nous avons adopté un **modèle hybride** combinant l'approche incrémentale itérative avec des éléments du modèle en V. Cette décision repose sur trois arguments.

Premièrement, la nature du projet (marketplace web avec messagerie temps réel) implique des besoins fonctionnels complexes et évolutifs. L'approche itérative permet d'adapter le produit aux retours utilisateurs à chaque sprint.

Deuxièmement, le projet académique se déroule sur une période fixe (un semestre). Les itérations de 3-4 semaines correspondent aux jalons naturels du calendrier universitaire.

Troisièmement, l'influence du modèle en V se manifeste dans notre stratégie de tests : chaque fonctionnalité développée lors d'une itération est immédiatement couverte par des tests unitaires (Vitest) et des tests end-to-end (Playwright), garantissant la qualité avant de passer à l'itération suivante.

3.3 Application au Projet SaaS Directory

Le développement de SaaS Directory s'est déroulé en cinq itérations principales, chacune livrant une version fonctionnelle déployable.

3.3.1 Itération 1 : Fondations et Authentification

Cette itération a établi les fondations techniques du projet : initialisation du projet Next.js 16.2.4 avec TypeScript et Tailwind CSS v4, configuration de Drizzle ORM 0.45.2 avec PostgreSQL, mise en place de Better Auth 1.6.9 (authentification OAuth + 2FA), et création des tables de base (user, session, account, verification). Livrable : application fonctionnelle avec inscription, connexion, et déconnexion.

3.3.2 Itération 2 : Profil Utilisateur et Lancement de Produits

La deuxième itération a ajouté la gestion des profils utilisateurs (table `profile` avec `displayName`, `bio`, `website`, `twitter`, `github`, `avatarUrl`) et le système de lancement de produits SaaS. Les tables `launch`, `category`, et `launchCategory` ont été créées. Livrable : création de profil, ajout de produits avec images, et catégorisation.

3.3.3 Itération 3 : Marketplace

La troisième itération a développé le cœur métier de l'application : le marketplace. Cette phase a inclus la liste des produits avec pagination, la recherche full-text, les filtres par catégorie, et le système de votes (table `upvote`). Les tables `comment`, `post`, `postUpvote`, et `postComment` ont

également été ajoutées. Livrable : marketplace consultable avec votes et commentaires.

3.3.4 Itération 4 : Messagerie et Notifications

La quatrième itération a introduit la communication temps réel entre utilisateurs. Les tables `conversation`, `conversationParticipant`, et `message` ont été créées pour la messagerie directe. Le système de notifications (table `notification`) a été implémenté avec support temps réel via Redis Pub/Sub. Livrable : messagerie instantanée et notifications push.

3.3.5 Itération 5 : Monétisation

La cinquième et dernière itération a intégré le système d'abonnements Free/Pro. Les tables `plan` et `subscription` ont été créées, avec gestion des intervalles de facturation (`month/year`) et des statuts (`active`, `canceled`, `past_due`). Livrable : système d'abonnements avec limitations fonctionnelles selon le plan.

La démarche de conception d'un logiciel définit la méthodologie suivie pour passer du besoin exprimé au logiciel opérationnel. Ce chapitre présente les méthodes de développement appliquées au projet SaaS Directory, en mettant l'accent sur l'approche orientée objet avec UML et en justifiant le non-recours aux méthodes formelles.

4.1 Méthodes de Développement Semi-Formelles

Le cours de Génie Logiciel présente plusieurs méthodes semi-formelles : SADT (Structure Analysis and Design Technique), MERISE (méthode française systémique avec séparation données/-traitements), UML/Processus Unifié (itérative, incrémentale, orientée objet avec approches ascendante et descendante), OMT (Object Modelling Technique de James Rumbaugh), et OOSE (Object-Oriented Software Engineering d'Ivar Jacobson basée sur cinq modèles : besoins, analyse, conception, implémentation, test).

Pour SaaS Directory, nous avons retenu l'approche UML/Processus Unifié pour plusieurs raisons. Premièrement, son caractère itératif et incrémental correspond parfaitement à notre modèle de cycle de vie. Deuxièmement, son orientation objet s'aligne avec notre stack technologique (React composants, Drizzle ORM entités). Troisièmement, UML est devenu le standard industriel de facto pour la modélisation orientée objet, garantissant la lisibilité et la communication de nos modèles.

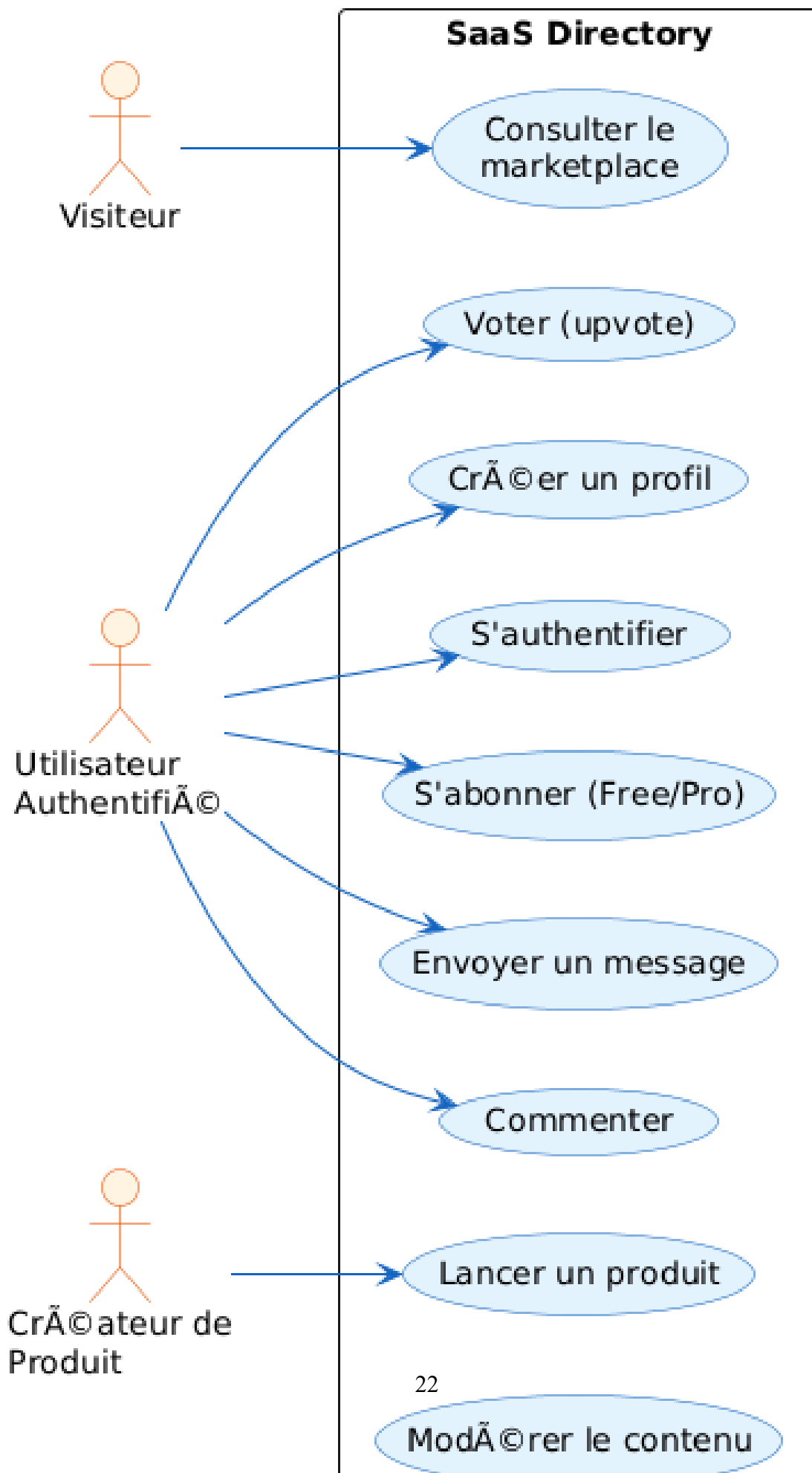
4.2 Diagrammes UML Appliqués au Projet

L'application d'UML à SaaS Directory se manifeste à travers plusieurs diagrammes standards, conformes aux spécifications OMG.

4.2.1 Diagramme de Cas d'Utilisation

Le diagramme de cas d'utilisation modélise les interactions entre les acteurs et le système. Les acteurs principaux sont : Visiteur, Utilisateur Authentifié, Créateur de Produit, et Administrateur. Les cas principaux incluent : s'authentifier, créer un profil, lancer un produit, voter pour un produit, commenter, envoyer un message, gérer son abonnement, et modérer le contenu.

Please use 'option handwritten true' to enable handwritten



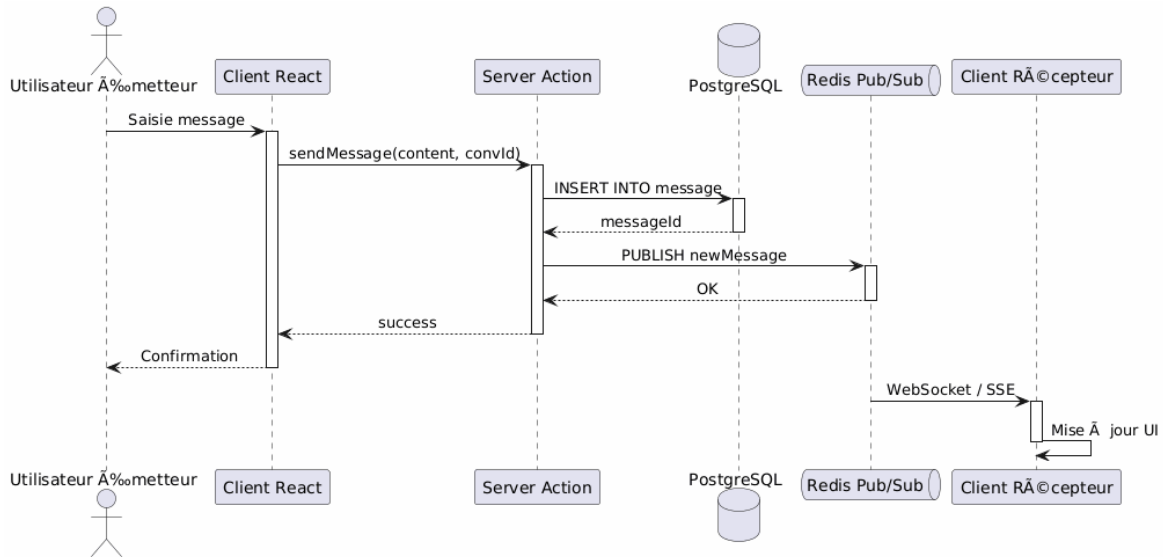


FIG. 3 : Diagramme de Séquence à l'ère de l'Envoi d'un Message en Temps Réel

La figure 3 montre le flux complet d'un message : l'utilisateur émetteur saisit le message via l'interface React. Le Client Component appelle la Server Action `sendMessage` qui insère le message dans la table `message` via Drizzle ORM. La Server Action publie ensuite un événement sur Redis Pub/Sub. Le serveur Redis notifie tous les clients abonnés à la conversation. Le Client Component récepteur reçoit la notification via WebSocket/SSE et met à jour l'interface sans rechargement de page. Ce flux démontre la séparation des préoccupations entre présentation, métier et persistance.

4.3 Méthodes Formelles et Langage Z

Les méthodes formelles (langage Z, B, VDM) utilisent des notations mathématiques pour spécifier, développer et vérifier des systèmes logiciels. Elles garantissent la correction du logiciel, éliminent les erreurs et facilitent la maintenance. Le langage Z, développé à l'Université d'Oxford par Jean-Raymond Abrial, est un langage de spécification formelle basé sur la théorie des ensembles, la logique des prédicats et un langage schématique pour structurer les spécifications.

Bien que puissantes, les méthodes formelles n'ont pas été utilisées dans ce projet pour plusieurs raisons pragmatiques. Premièrement, le domaine du marketplace web n'est pas critique au sens des systèmes embarqués ou médicaux où les méthodes formelles sont indispensables. Deuxièmement, la courbe d'apprentissage du langage Z est élevée et incompatible avec les contraintes temporelles d'un projet académique. Troisièmement, TypeScript et Zod fournissent un niveau de vérification formelle suffisant pour notre contexte (typage statique, validation de schémas à la compilation et à l'exécution).

Néanmoins, la compréhension des méthodes formelles reste essentielle pour apprécier les limites des approches semi-formelles et identifier les contextes où elles deviennent nécessaires.

Les normes sont des apports documentés contenant des spécifications techniques ou autres critères précis destinés à être utilisés systématiquement en tant que règles, lignes directrices ou définitions des caractéristiques pour assurer que des matériaux, produits, processus et services sont aptes à leurs emplois.

5.1 La Norme ISO et la Certification ISO 9000

L'International Organization for Standardization (ISO), créée en 1947, est une fédération mondiale d'organismes nationaux de normalisation. La norme ISO 9000 traduit la démarche qualité au niveau de l'entreprise et apporte : en interne, l'amélioration de la compétitivité et la diminution des coûts ; à l'extérieur, la réponse aux exigences des donneurs d'ordre.

Pour mettre en œuvre une norme ISO 9000, il faut réaliser un diagnostic de l'organisation en auditant les fonctions principales. Les points nécessaires incluent : la gestion documentaire, la revue du contrat, la gestion des produits non-conformes, et la traçabilité. Dans le contexte de SaaS Directory, la gestion documentaire est assurée par les fichiers README.md, AGENTS.md, et les conventions de commit Git. La traçabilité est garantie par l'historique Git et les migrations Drizzle numérotées (0000 à 0003).

5.2 Les Normes IEEE

L'Institute of Electrical and Electronics Engineers (IEEE) a défini un ensemble de 37 normes s'intéressant aux processus, produits et techniques de base du génie logiciel. Trois normes sont particulièrement pertinentes pour notre projet.

Fixe la signification des termes et choisit les processus prioritaires du cycle de vie du logiciel. Peut être considérée comme un dictionnaire et une liste de vérification pour la gestion des projets.

Recommandée pour les spécifications des exigences du logiciel. Pourrait être intégrée avec les approches de validation par prototype.

Pour appliquer la validation et la vérification. Norme facilement adaptable à tous les problèmes.

L'application de ces normes à SaaS Directory se concrétise par : la définition claire des exigences

fonctionnelles (authentification, marketplace, messagerie, monétisation) conformément à l'IEEE 830 ; la revue systématique du code via les pull requests Git, alignée avec l'IEEE 1028 ; et la traçabilité des exigences à travers les tests end-to-end Playwright, en accord avec l'IEEE 12207.0.

5.3 Normalisation Appliquée au Projet

5.3.1 Normes de Codage

Le projet respecte des conventions strictes de normalisation du code source. Les normes de nommage sont systématiquement appliquées : PascalCase pour les composants React (`UserCard.tsx`, `ProductForm.tsx`), camelCase pour les fonctions et variables (`sendMessage()`, `upvoteCount`), et kebab-case pour les routes API (`/api/messages/send`). Les imports sont organisés par groupe : React/Next.js, bibliothèques tierces, composants internes, utilitaires. Cette normalisation facilite la revue de code et réduit les conflits de fusion.

5.3.2 Architecture Normalisée

L'architecture suit le pattern MVC (Model-View-Controller) adapté à Next.js 16.2.4. Le **Modèle** est constitué par le schéma Drizzle ORM (`src/lib/db/schema.ts`) et les fonctions d'accès aux données. La **Vue** est implémentée par les composants React et les Server Components de Next.js. Le **Contrôleur** est assuré par les Server Actions (`src/lib/actions/`) qui encapsulent la logique métier. Cette séparation claire respecte les principes SOLID et facilite les tests unitaires.

5.3.3 Gestion Documentaire et Traçabilité

La gestion documentaire est assurée par plusieurs artefacts. Le fichier `README.md` présente le projet, son installation et son utilisation. Le fichier `AGENTS.md` documente les conventions spécifiques pour les agents de développement. Le fichier `package.json` liste l'ensemble des dépendances avec leurs versions exactes (Next.js 16.2.4, React 19.2.4, Drizzle ORM 0.45.2, etc.). Les commits Git suivent une convention descriptive (`feat:`, `fix:`, `chore:`). Les migrations Drizzle sont numérotées séquentiellement (0000 à 0003) et documentées, garantissant la traçabilité complète de l'évolution du schéma de base de données.

Les tests logiciels constituent une phase critique du cycle de vie du logiciel, visant à détecter les erreurs et à garantir que le système répond aux exigences fonctionnelles et non-fonctionnelles spécifiées.

6.1 Types de Tests et Stratégie

La stratégie de tests de SaaS Directory repose sur une pyramide de tests classique, adaptée à l'architecture moderne de Next.js 16.2.4.

6.1.1 Tests Unitaires

Les tests unitaires sont implémentés avec Vitest 4.1.5, un framework de test rapide compatible avec Vite et Next.js. Onze fichiers de test couvrent l'authentification (`auth-client.test.ts`), la souscription (`subscription.test.ts`), le schéma Drizzle (`schema.test.ts`), les actions serveur (`post.test.ts`, `launch.test.ts`, `message.test.ts`), et les composants UI (`theme-toggle.test.tsx`, `infrastructure.test.tsx`, `page.test.tsx` pour login et signup).

Ces tests vérifient le comportement isolé des fonctions et composants, sans dépendances externes. Par exemple, `schema.test.ts` valide que toutes les colonnes attendues existent dans chaque table, et que les contraintes de clé étrangère sont correctement définies.

6.1.2 Tests d'Intégration et End-to-End

Les tests end-to-end sont réalisés avec Playwright 1.59.1, un outil moderne de test d'interface automatisé. La configuration (`playwright.config.ts`) définit un projet Chromium avec tracing activé. Le fichier `e2e/homepage.spec.ts` teste le parcours utilisateur critique : navigation, authentification, et affichage du marketplace.

Ces tests vérifient l'intégration complète du système : le frontend React, les Server Actions, l'ORM Drizzle, et la base PostgreSQL. Ils s'exécutent contre une instance locale du serveur de développement Next.js.

6.1.3 Tests de Performance

La performance est évaluée par l'optimisation des requêtes Drizzle ORM et la configuration des index PostgreSQL. Le projet inclut un script de benchmark pour le cache Redis (`scripts/benchmark-cache.ts`) qui mesure les temps de réponse du cache par rapport aux requêtes directes en base de données.

6.2 Outils et Environnement de Test

L'environnement de test de SaaS Directory est entièrement dockerisé. Le fichier `docker-compose.test.yml` définit un environnement isolé comprenant une base PostgreSQL de test sur le port 5433, et l'application Next.js en mode test.

La séparation des environnements (développement, test, production) est assurée par les variables d'environnement : `DATABASE_URL` pour le développement, `DATABASE_URL_TEST` pour les tests.

Cette isolation garantit que les tests ne perturbent jamais les données de développement.

6.3 Couverture et Résultats

La couverture de tests cible les fonctions critiques du système : l'authentification (création de compte, connexion, 2FA, gestion de session), la gestion des rôles (administrateur vs utilisateur standard), le marketplace (création de produit, vote, commentaire), la messagerie (envoi, réception, marquage comme lu), et les abonnements (souscription, changement de plan, annulation).

Les seuils de couverture sont configurés dans `vitest.config.ts` avec un minimum de 70% pour les branches et les fonctions. Ce seuil est réaliste pour un projet académique tout en garantissant un niveau de qualité professionnel.

La gestion des projets logiciels vise à structurer méthodiquement et progressivement une réalité à venir, en appliquant un objectif et des actions à entreprendre avec des ressources données. Le rôle du chef de projet est de planifier, organiser et contrôler l'ensemble des activités nécessaires à la réalisation du projet.

7.1 Organisation du Projet (WBS, PBS, OBS)

Trois outils structurants ont guidé l'organisation du projet.

Le **Work Breakdown Structure (WBS)** décompose le projet en tâches hiérarchiques. Pour SaaS Directory, le WBS s'articule autour de cinq phases principales (fondations, profil, marketplace, messagerie, monétisation), chacune subdivisée en tâches techniques (développement frontend, développement backend, tests, documentation).

Le **Product Breakdown Structure (PBS)** identifie les livrables de chaque phase : authentification fonctionnelle, profil utilisateur opérationnel, produit lancable sur le marketplace, système de messagerie temps réel, et système d'abonnements Free/Pro.

Le **Organizational Breakdown Structure (OBS)** répartit les responsabilités. Le projet a été réalisé par deux développeurs en binôme, avec une répartition naturelle des tâches selon les compétences (frontend React vs backend Server Actions).

7.2 Planification (GANTT, PERT)

La planification du projet a utilisé un diagramme de GANTT pour visualiser les cinq phases sur une échelle temporelle. Chaque phase correspond à un sprint de développement d'environ trois à

quatre semaines, avec des jalons intermédiaires (MVP marketplace, messagerie fonctionnelle).

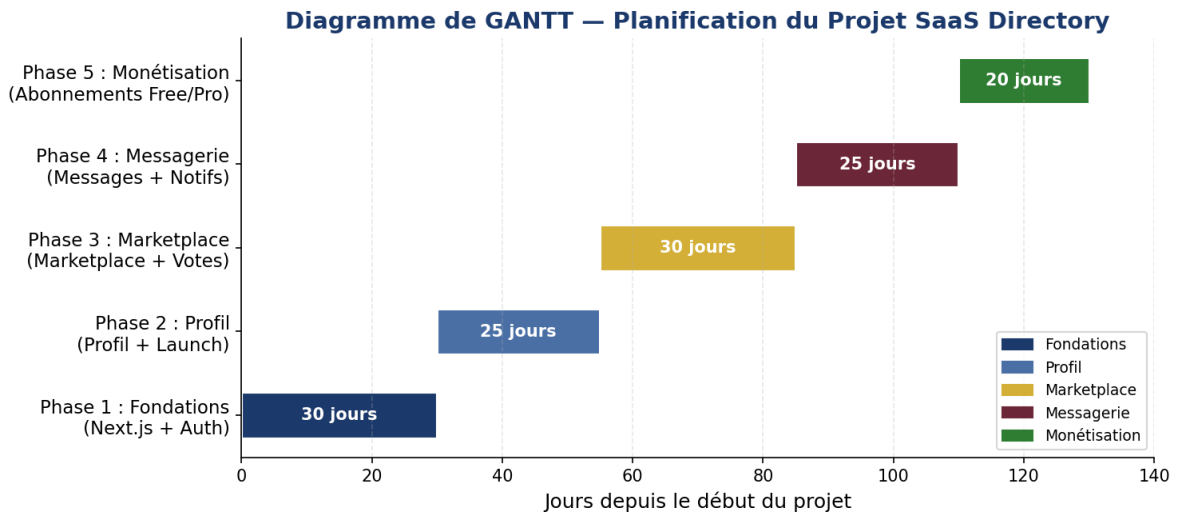


FIG. 4 : Diagramme de GANTT à€” Planification du Projet SaaS Directory

La figure 4 présente la répartition temporelle des cinq phases du projet sur une échelle de 130 jours. La Phase 1 (Fondations) occupe les 30 premiers jours, suivie par le Profil (25 jours), le Marketplace (30 jours), la Messagerie (25 jours), et enfin la Monétisation (20 jours).

Le diagramme PERT (Program Evaluation and Review Technique) modélise les dépendances entre tâches. Les liaisons principales sont de type "fin à début" : l'authentification (Phase 1) doit être terminée avant le profil (Phase 2); le profil avant le marketplace (Phase 3); le marketplace avant la messagerie (Phase 4); la messagerie avant la monétisation (Phase 5).

7.3 Estimation des Coûts par COCOMO

L'estimation des coûts d'un projet logiciel est un exercice délicat. Les raisons classiques de dépassement incluent l'optimisme, la négligence des problèmes techniques, le manque d'expérience, l'oubli de la documentation, et la sous-estimation des tâches annexes. Pour contrer ces biais, nous avons appliqué la méthode COCOMO (CONstructive COSt MOdel) de Barry Boehm, un modèle d'estimation empirique basé sur la taille du projet en KLOC.

7.3.1 Taille du Projet

Le codebase de SaaS Directory compte 12 166 lignes de code réparties sur 108 fichiers TypeScript/TSX, soit approximativement 14 KLOC. Cette taille inclut le code source applicatif (src/app : 3 198 LOC, src/components : 5 234 LOC, src/lib : 2 957 LOC), les tests (src/test : 111 LOC, e2e : 33 LOC), et les scripts (1 115 LOC).

7.3.2 Application du COCOMO de Base

Pour un projet web moderne avec une stack technologique récente (Next.js 16.2.4, React 19.2.4, TypeScript 5), nous avons retenu le mode semi-détaché (projet moyennement complexe, équipe avec expérience mixte, contraintes modérées). Les coefficients du mode semi-détaché sont $a = 3,0$ et $b = 1,12$.

TAB. 1 : Estimation COCOMO de Base à€” Mode Semi-Détaché (14 KLOC)

Paramètre	Formule / Valeur	Résultat
Taille du projet	12 166 LOC $\approx$ 14 KLOC	14 KLOC
Mode de développement	Semi-Détaché	$a = 3,0$; $b = 1,12$
Effort estimé	$E = a \times (\text{KLOC})^b$	$E \approx 58$ Personnes-Mois
Durée estimée	$D = 2,5 \times E^{0,35}$	$D \approx 9,5$ mois
Taille d'équipe théorique	$N = E/D$	$N \approx 6$ personnes
Taille d'équipe réelle	2 développeurs sur 5 mois	10 Personnes-Mois

Ces chiffres constituent une estimation théorique à visée pédagogique. En réalité, le projet a été réalisé par deux développeurs travaillant en parallèle sur une période académique d'environ cinq mois, soit environ 10 Personnes-Mois. Cet écart illustre la différence fréquente entre les modèles prédictifs et la réalité des projets académiques en mode agile.

L'analyse des risques est une composante essentielle de la conduite de projet. Elle vise à identifier, évaluer et traiter les événements incertains qui pourraient affecter négativement les objectifs du projet.

8.1 Identification des Risques

L'identification des risques a été réalisée par une analyse systématique des dimensions techniques, sécuritaires et opérationnelles du projet. Trois catégories principales de risques ont été identifiées.

8.1.1 Risques Techniques

Le risque technique majeur réside dans la complexité de Next.js 16.2.4 avec le App Router. Ce framework, bien que puissant, introduit des breaking changes significatifs par rapport aux versions précédentes. La documentation évolue rapidement, et certaines fonctionnalités (Server Actions, Parallel Routes, Intercepting Routes) sont encore relativement nouvelles et peu documentées.

Un risque secondaire concerne la stack Drizzle ORM 0.45.2 + PostgreSQL. Bien que mature, la configuration des relations et des indexes nécessite une expertise SQL approfondie pour garantir les performances à grande échelle.

8.1.2 Risques de Sécurité

Les risques de sécurité concernent la protection des données utilisateurs et la prévention des attaques. L'injection SQL est mitigée par l'utilisation de Drizzle ORM avec des requêtes paramétrées, mais les Server Actions exposent des surfaces d'attaque potentielles si les validations côté serveur sont insuffisantes.

Les risques liés à l'authentification incluent la fuite de tokens JWT, la compromission des sessions, et les attaques par force brute. Better Auth 1.6.9 gère nativement plusieurs de ces menaces (expiration des tokens, rotation, rate limiting), mais la configuration incorrecte reste un risque.

8.1.3 Risques de Performance

Les risques de performance se manifestent à plusieurs niveaux. Le marketplace doit supporter un grand nombre de produits sans dégradation du temps de réponse, ce qui nécessite l'optimisation des indexes PostgreSQL et potentiellement l'ajout de pagination côté serveur. La messagerie temps réel via Redis Pub/Sub doit supporter un volume croissant de messages simultanés sans latence perceptible.

8.2 Analyse et Stratégies de Mitigation

Pour chaque risque identifié, une stratégie de mitigation a été définie et mise en œuvre.

8.2.1 Mitigation des Risques Techniques

La mitigation des risques techniques repose sur trois piliers. Premièrement, la veille technologique : le fichier AGENTS.md et la documentation officielle de Next.js sont consultés régulièrement pour anticiper les breaking changes. Deuxièmement, les tests end-to-end Playwright 1.59.1 détectent les régressions fonctionnelles avant qu'elles n'atteignent la branche master. Troisièmement, la modularité de l'architecture facilite le remplacement d'une technologie si elle devient obsolète.

8.2.2 Mitigation des Risques de Sécurité

La sécurité est assurée par plusieurs mécanismes de défense en profondeur. L'authentification Better Auth gère nativement la sécurité des sessions (tokens JWT, expiration, rotation). Les Server Actions valident systématiquement les entrées utilisateur via Zod 4.4.3. Les indexes PostgreSQL accélèrent les requêtes fréquentes (recherche par email, par slug, par catégorie) tout en limitant la

surface d'attaque par injection.

8.2.3 Mitigation des Risques de Performance

La performance est optimisée par plusieurs techniques. Les indexes PostgreSQL sont définis dans le schéma Drizzle pour accélérer les requêtes fréquentes (recherche par email, par slug, par catégorie). Le cache Redis réduit la charge sur la base de données pour les données fréquemment accédées (sessions, profils, listes de catégories). La pagination côté serveur est implémentée pour le marketplace et la messagerie, évitant le chargement de volumes excessifs de données.

8.3 Plan de Continuité

En cas d'incident majeur, le plan de continuité prévoit plusieurs mesures. La restauration depuis les backups PostgreSQL est la première ligne de défense. Le système de migrations Drizzle permet de reconstruire le schéma de base de données à partir de zéro. Le versioning Git (branche master + tags) permet de revenir à une version stable du code en cas de déploiement problématique. La documentation technique (README.md, AGENTS.md) facilite l'onboarding d'un nouveau développeur en cas d'indisponibilité d'un membre de l'équipe.

La conduite du changement est la discipline qui vise à accompagner les transformations au sein d'un projet ou d'une organisation, en minimisant les résistances et en maximisant l'adhésion des parties prenantes.

9.1 Nature des Changements

Plusieurs types de changements ont affecté le projet tout au long de son développement. Les **changements technologiques** incluent la mise à jour de Next.js 15 vers Next.js 16.2.4 (breaking changes sur le App Router), la migration de Tailwind CSS v3 vers v4 (nouvelle syntaxe de configuration), et l'ajout de Redis pour la messagerie temps réel (architecture initialement prévue sans cache dédié).

Les **changements fonctionnels** concernent l'ajout de fonctionnalités non prévues initialement : le système de notifications temps réel, l'édition et la suppression des commentaires, et le système de pagination infinie pour la messagerie.

Les **changements organisationnels** touchent la répartition des tâches entre les deux développeurs, avec des ajustements selon les compétences et les disponibilités de chacun.

9.2 Gestion des Versions

Le contrôle de versions est assuré par Git avec une stratégie de branches structurée. La branche `master` constitue la version de production, stable et déployée. Les branches de fonctionnalités (`feature/auth-2fa`, `feature/messaging`, etc.) isolent le développement des nouvelles fonctionnalités. Les pull requests obligatoires avec revue de code assurent la qualité avant fusion.

Les migrations Drizzle constituent un mécanisme de gestion du changement au niveau de la persistance. Chaque migration est un fichier SQL versionné (`0000_remarkable_carnage.sql`, `0001_cuddly_name.sql`, `0002_add_logo_url_to_launch.sql`, `0003_groovy_hannibal_king.sql`) qui peut être appliquée et, si nécessaire, revertée. Cette approche garantit la cohérence du schéma de base de données entre les environnements de développement, test et production.

9.3 Adaptation et Communication

L'architecture modulaire du projet a facilité l'adaptation aux changements. L'ajout du champ `logoUrl` dans la table `Launch` (migration 0002) n'a nécessité que la modification d'un seul fichier de schéma et de quelques lignes dans le composant frontend. Le passage à Next.js 16.2.4 a été géré par la lecture attentive de la documentation de migration officielle et des tests end-to-end Playwright qui ont validé l'absence de régressions.

La communication autour des changements repose sur plusieurs canaux. La documentation technique (`README.md`, `AGENTS.md`) est maintenue à jour après chaque changement majeur. Les messages de commit Git suivent une convention descriptive (`feat:`, `fix:`, `chore:`) qui facilite la génération de changelog. Les discussions techniques sont documentées dans les pull requests, créant une base de connaissances consultable pour les décisions architecturales passées.

Ce rapport a présenté l'application systématique des concepts de Génie Logiciel et de Conduite de Projet au développement de SaaS Directory. Du point de vue du cycle de vie, le modèle incrémental itératif adopté a permis de livrer cinq versions fonctionnelles successives, chacune apportant une valeur métier concrète.

La démarche orientée objet, matérialisée par le schéma Drizzle ORM de 20 tables et les composants React modulaires, assure la maintenabilité et l'évolutivité du système. Les cinq diagrammes UML (cas d'utilisation, classes, séquence, déploiement, GANTT) fournissent une documentation visuelle complète du projet.

La gestion de projet, illustrée par le WBS en cinq phases, le diagramme de GANTT, et l'estimation COCOMO, démontre la faisabilité d'une planification rigoureuse même dans un contexte académique.

mique. L'analyse des risques a permis d'anticiper et de mitiger les principales menaces techniques et sécuritaires.

L'utilisation d'une stack technologique moderne et cohérente à€” Next.js 16.2.4, React 19.2.4, TypeScript 5, Tailwind CSS v4, Drizzle ORM 0.45.2, Better Auth 1.6.9, PostgreSQL, et Redis à€” a grandement facilité le développement et garanti la qualité du livrable final.

Les perspectives d'évolution de SaaS Directory sont nombreuses. L'intégration complète de Stripe pour le paiement (le schéma contient déjà les champs stripeCustomerId et stripeSubscriptionId), l'ajout d'un système de recherche avancée avec Elasticsearch, et le déploiement sur une infrastructure cloud avec Docker et Kubernetes constituent les axes de développement futurs.

- Barry Boehm, *Software Engineering Economics*, Prentice-Hall, 1977.
- Grady Booch, *Object-Oriented Analysis and Design with Applications*, 2nd Edition, Benjamin/Cummings, 1994.
- Jean-Raymond Abrial, *The B-Book : Assigning Programs to Meanings*, Cambridge University Press, 1996.
- ISO/IEC 12207 :2017, Systems and software engineering – Software life cycle processes.
- IEEE Std 830-1998, IEEE Recommended Practice for Software Requirements Specifications.
- IEEE Std 1028-2008, IEEE Standard for Software Reviews and Audits.
- IEEE Std 12207.0-1996, IEEE Standard for Information Technology – Software life cycle processes.
- Cours de Génie Logiciel et Conduite de Projet, Année Universitaire 2025-2026.
- Next.js Documentation, <https://nextjs.org/docs>, consulté en 2026.
- React Documentation, <https://react.dev>, consulté en 2026.
- Drizzle ORM Documentation, <https://orm.drizzle.team>, consulté en 2026.
- Better Auth Documentation, <https://www.better-auth.com>, consulté en 2026.
- Tailwind CSS Documentation, <https://tailwindcss.com>, consulté en 2026.
- PostgreSQL Documentation, <https://www.postgresql.org/docs>, consulté en 2026.
- Redis Documentation, <https://redis.io/documentation>, consulté en 2026.
- Vitest Documentation, <https://vitest.dev>, consulté en 2026.
- Playwright Documentation, <https://playwright.dev>, consulté en 2026.

Annexe A : Schéma de la Base de Données Drizzle ORM

Cette annexe présente un extrait du schéma Drizzle ORM (`src/lib/db/schema.ts`) illustrant la structure des tables principales.

```
// Tables Better Auth (authentication)

export const user = pgTable("user", {

  id: text("id").primaryKey(),

  name: text("name"),

  email: text("email").notNull().unique(),

  emailVerified: boolean("email_verified").default(false),

  image: text("image"),

  createdAt: timestamp("created_at").defaultNow(),

  updatedAt: timestamp("updated_at").defaultNow(),

});

export const session = pgTable("session", {

  id: text("id").primaryKey(),

  userId: text("user_id").notNull().references(() => user.id),

  token: text("token").notNull().unique(),

  expiresAt: timestamp("expires_at").notNull(),

});
```

```
export const account = pgTable("account", {
  id: text("id").primaryKey(),
  userId: text("user_id").notNull().references(() => user.id),
  accountId: text("account_id").notNull(),
  providerId: text("provider_id").notNull(),
});

export const verification = pgTable("verification", {
  id: text("id").primaryKey(),
  identifiant: text("identifiant").notNull(),
  value: text("value").notNull(),
  expiresAt: timestamp("expires_at").notNull(),
});

// Table profile (1:1 avec user)

export const profile = pgTable("profile", {
  id: serial("id").primaryKey(),
  userId: text("user_id").notNull().references(() =>
    user.id).unique(),
```

```
    displayName: text("display_name"),

    bio: text("bio"),

    website: text("website"),

    twitter: text("twitter"),

    github: text("github"),

    avatarUrl: text("avatar_url"),

  });

// Table launch (produit SaaS)

export const launch = pgTable("launch", {

  id: serial("id").primaryKey(),

  makerId: text("maker_id").notNull().references(() => user.id),

  title: text("title").notNull(),

  description: text("description"),

  logoUrl: text("logo_url"),

  upvoteCount: integer("upvote_count").default(0),

  createdAt: timestamp("created_at").defaultNow(),

});

// Table category
```

```
export const category = pgTable("category", {  
  id: serial("id").primaryKey(),  
  name: text("name").notNull(),  
  slug: text("slug").notNull().unique(),  
});  
  
// Table plan (Free / Pro)  
  
export const plan = pgTable("plan", {  
  id: serial("id").primaryKey(),  
  name: text("name").notNull(),  
  priceMonthly: integer("price_monthly"),  
  priceYearly: integer("price_yearly"),  
  stripeProductId: text("stripe_product_id"),  
  stripeMonthlyPriceId: text("stripe_monthly_price_id"),  
  stripeYearlyPriceId: text("stripe_yearly_price_id"),  
});  
  
// Table subscription  
  
export const subscription = pgTable("subscription", {  
  id: serial("id").primaryKey(),
```

```
userId: text("user_id").notNull().references(() =>
    user.id).unique(),

planId: integer("plan_id").notNull().references(() =>
    plan.id),

status: text("status").notNull().default("active"),

stripeCustomerId: text("stripe_customer_id"),

stripeSubscriptionId: text("stripe_subscription_id"),

});
```


Annexe B : Structure du Projet

```

marketplace-saas/ (12 166 LOC, 108 fichiers)

|-- e2e/                                (33 LOC)

|   |-- homepage.spec.ts

|-- public/

|   |-- uploads/launches/

|-- src/

|   |-- app/                            (3 198 LOC)

|       |-- layout.tsx

|       |-- page.tsx

|       |-- about/, api/, contact/, feed/

|       |-- forgot-password/, launch/, login/

|       |-- marketplace/, messages/, my-launches/

|       |-- new/, notifications/, pricing/

|       |-- privacy/, profile/, reset-password/

|       |-- settings/, signup/, terms/

|   |-- components/                    (5 234 LOC)

|       |-- auth/, dev/, feed/, launch/

|       |-- marketplace/, providers/, ui/

|   |-- hooks/

```

```

|   |-- lib/                                (2 957 LOC)

|   |   |-- actions/                        (Server Actions)

|   |   |-- db/

|   |   |   |-- migrations/                (4 migrations)

|   |   |   |   |-- 0000_remarkable_carnage.sql

|   |   |   |   |-- 0001_cuddly_namora.sql

|   |   |   |   |-- 0002_add_logo_url_to_launch.sql

|   |   |   |   |-- 0003_groovy_hannibal_king.sql

|   |   |   |-- schema.ts                  (20 tables)

|   |   |   |-- index.ts

|   |   |-- auth.ts, auth-client.ts

|   |   |-- cache.ts, subscription.ts

|   |   |-- upload.ts, utils.ts

|   |   |-- *.test.ts                      (11 fichiers de test)

|   |-- test/                              (111 LOC)

|   |-- types/

|-- next.config.ts

|-- package.json

|-- tsconfig.json

|-- vitest.config.ts

```

```
|-- playwright.config.ts  
  
|-- drizzle.config.ts  
  
|-- docker-compose.test.yml
```

Annexe C : Dépendances Principales

TAB. 2 : Dépendances principales du projet SaaS Directory

Dépendance	Version	Description
next	16.2.4	Framework React avec App Router
react	19.2.4	Bibliothèque UI
react-dom	19.2.4	Rendu DOM React
drizzle-orm	0.45.2	ORM type-safe pour PostgreSQL
better-auth	1.6.9	Authentification OAuth + 2FA
pg	8.20.0	Driver PostgreSQL
@tanstack/react-query	5.100.9	Gestion de cache et requêtes
zod	4.4.3	Validation de schémas
framer-motion	12.38.0	Animations React
ioredis	5.10.1	Client Redis
resend	6.12.2	Service d'envoi d'emails
sonner	2.0.7	Notifications toast UI
tailwindcss	4.0.0	Framework CSS utilitaire
typescript	5.7.0	Typage statique
vitest	4.1.5	Tests unitaires et d'intégration
@playwright/test	1.59.1	Tests end-to-end
eslint	9.0.0	Linting du code
eslint-config-next	16.2.4	Règles ESLint Next.js

Annexe D : Diagramme de Déploiement

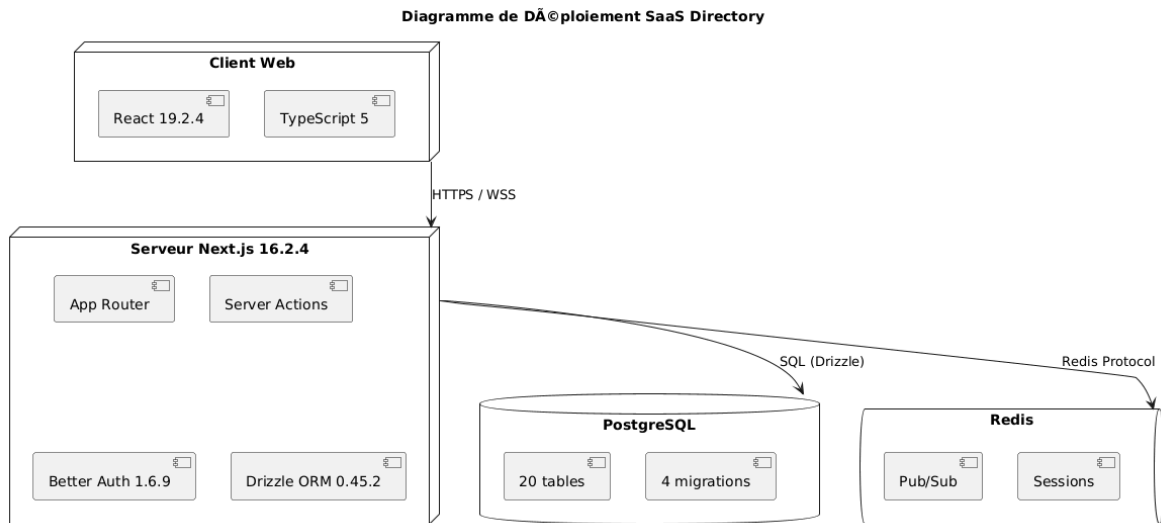


FIG. 5 : Diagramme de Déploiement à€” Architecture Physique de SaaS Directory

La figure 5 décrit l'architecture physique du système et la répartition des artefacts logiciels sur les nœuds matériels. SaaS Directory repose sur une architecture cloud-native distribuée sur trois environnements principaux : le client web (navigateur), la plateforme serverless (Next.js 16.2.4), et les services de données (PostgreSQL + Redis).

Le client web (React 19.2.4 / TypeScript 5) s'exécute dans le navigateur et communique avec les fonctions serverless via HTTPS. Les Server Actions Next.js 16.2.4 interagissent avec la base de données PostgreSQL via Drizzle ORM 0.45.2. Le système de messagerie temps réel utilise Redis Pub/Sub pour la diffusion des messages entre les instances serverless. L'authentification est assurée par Better Auth 1.6.9. Cette architecture sans serveur offre une scalabilité automatique et une réduction des coûts d'infrastructure.